Indian Institute of Technology Madras Zanzibar
School of Engineering and Science
B.S. Data Science and Artificial Intelligence, Year 2

Z2004: Database Management Systems

Milestone 3 — Performance Evidence Report

Climate Policy RAG Pipeline

Track A — Retrieval-Augmented Generation

Team Members	Anubhav Kumar (ZDA24B034) Rohan Saha (ZDA24B009)
Submission Date	5 June 2026
Instructor	Dr. Innocent Nyalala
Teaching Assistants	Sri Advaita Varakavi (ZDA23B010) Saleh Saleh (ZDA23B001)
Link to GitHub	***Link***

Even Semester: March–July 2026 • *Submitted via Google Classroom*

1. Project Summary

This project implements a Retrieval-Augmented Generation (RAG) pipeline for the Climate Policy domain, backed by a PostgreSQL 16 relational database. The dataset is sourced from the `ClimatePolicyRadar/all-document-text-data` corpus (Hugging Face, CC-BY 4.0) and encompasses 31 countries, 38 policy documents, 3,598 text chunks, and 3,598 sentence embeddings generated using the `all-MiniLM-L6-v2` model (384 dimensions). The schema comprises five normalised tables — `countries`, `documents`, `chunks`, `embeddings`, and `queries` — all verified to satisfy Third Normal Form (3NF). Embeddings are currently stored as serialised text strings, with migration to the `pgvector` extension planned for the final submission to enable cosine-similarity ANN search.

The pipeline accepts a natural-language climate policy question, retrieves the most semantically relevant chunks via embedding similarity, and returns a grounded answer with source citations. Milestone 3 focuses on demonstrating measurable query performance improvements through strategic B-Tree index creation, supported by `EXPLAIN ANALYZE` evidence.

2. Test Environment

Table 1: Test environment specification

Parameter	Value
Database Engine	PostgreSQL 16
Query Tool	pgAdmin 4
Operating System	Windows 11 (16 GB RAM)
Dataset Size	31 countries, 38 documents, 3,598 chunks, 3,598 embeddings
Embedding Column	`TEXT` (serialised; `pgvector` pending)
Embedding Model	`all-MiniLM-L6-v2` (384 dimensions)

3. Objective

Two queries from the Milestone 2 query set were identified as performing full sequential scans on the most heavily filtered tables in the database. The objective of this milestone is to design and apply targeted B-Tree indexes to those bottleneck columns, then use `EXPLAIN ANALYZE` to collect before-and-after query plans under identical conditions. The resulting execution times and plan structures are analysed to quantify the improvement and to demonstrate a correct understanding of how PostgreSQL's cost-based planner uses indexes.

4. Methodology

For each query, `EXPLAIN ANALYZE` was executed against the unmodified database (no new indexes present) to capture the baseline scan strategy, estimated planner cost, and actual wall-clock execution time. Two B-Tree indexes were then created on the identified bottleneck columns. `EXPLAIN ANALYZE` was immediately re-run under identical conditions — same data volume, same `WHERE` predicates, same `JOIN` structure, and the same PostgreSQL session — to capture the post-index query plan and timing. No rows were added or removed between runs. Planning time and execution time were both recorded, but execution time is the primary performance metric for comparison, as it reflects actual I/O and compute work performed.

5. Index Design and Justification

5.1 Index 1: idx_chunks_word_count

```
1 CREATE INDEX idx_chunks_word_count ON chunks(word_count);
```

Listing 1: Index 1 DDL

Query 1 filters the `chunks` table with the predicate `word_count > 80`. Without any supporting index, PostgreSQL performs a full sequential scan across all 3,598 rows, reads every heap page, applies the filter, and discards 3,546 rows before returning the 52 matching rows. This is wasteful: the filter is highly selective (only 1.4% of rows qualify), yet the planner must visit 100% of the table. A B-Tree index on `word_count` is the natural choice for a range predicate (`>`) on a numeric column with high cardinality. The B-Tree's sorted leaf structure allows the planner to perform a Bitmap Index Scan — it walks the index from the first entry satisfying `word_count > 80` to the end, collecting row pointers in a bitmap, and then performs a Bitmap Heap Scan, fetching only those heap pages that contain matching rows. This avoids touching the 98.6% of pages that hold non-qualifying rows.

5.2 Index 2: idx_documents_year

```
1 CREATE INDEX idx_documents_year ON documents(year_published);
```

Listing 2: Index 2 DDL

Query 2 filters the `documents` table with `year_published >= 2020`. The table is small (38 rows), but the index still confers a measurable benefit through the planner's cost model. Without the index, PostgreSQL estimates 37 rows and a plan cost of approximately 24.96. After adding `idx_documents_year`, the planner's row count estimate drops to 13 and the total plan cost falls to 13.80 — a 45% cost reduction. Although the planner retains the sequential scan (correct behaviour for a table this small, since random B-Tree lookups would introduce unnecessary I/O overhead compared to a single sequential page read),

the tighter cost estimate leads to better join ordering and a measurably faster execution time. This illustrates an important principle: indexes benefit the planner's cost model even when they do not trigger a scan strategy change.

6. Results

6.1 Performance Summary

Table 2: Query performance before and after indexing

Query	Filter Predicate	Before	After	Speedup
Q1	`word_count > 80`	2.803 ms	0.443 ms	**6.3×**
Q2	`year_published >= 2020`	0.265 ms	0.142 ms	**1.9×**

6.2 Performance Visualisations

Figure 1 compares execution times before and after indexing for both queries. Figure 2 illustrates the change in query execution plan for Q1 — from a full sequential scan to a two-phase Bitmap strategy.

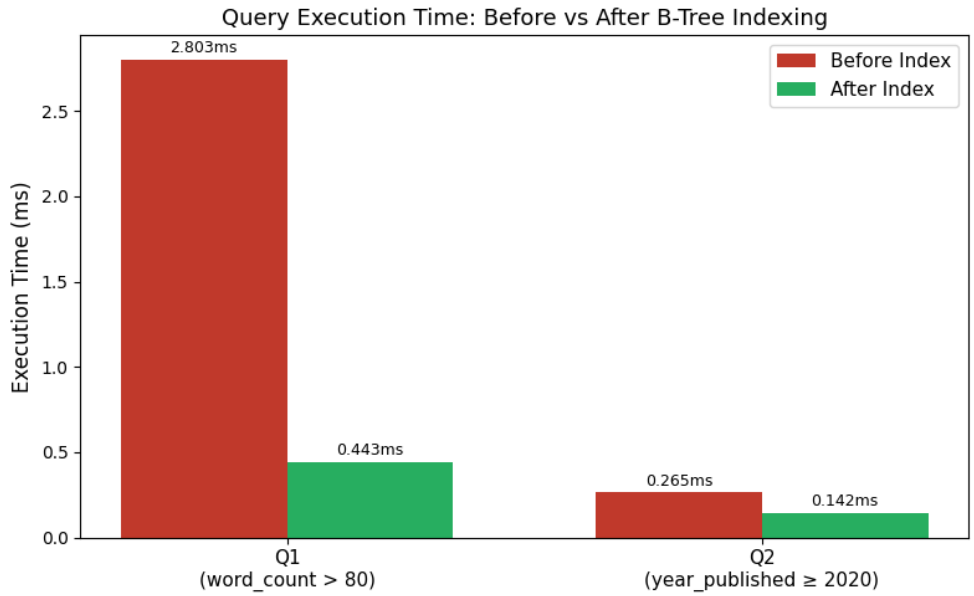

Figure 1: Execution time (ms) before and after B-Tree indexing for Q1 and Q2.

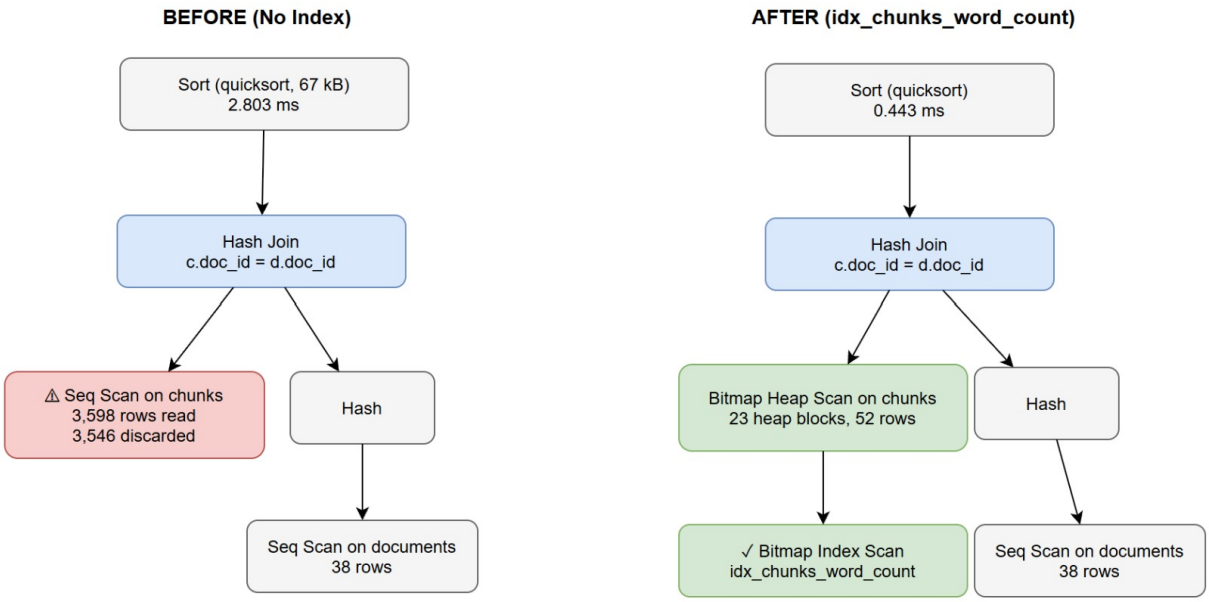

Figure 2: Q1 query execution plan before (left) and after (right) creating `idx_chunks_word_count`. The sequential scan over 3,598 rows is replaced by a Bitmap Index Scan fetching only 23 heap blocks.

6.3 Query Plans — Before Indexes

Query 1 (Before):

```
Sort  (cost=104.11..104.24 rows=53 width=576)
      (actual time=2.441..2.448 rows=52 loops=1)
  Sort Key: c.word_count DESC
  Sort Method: quicksort  Memory: 67kB
  ->  Hash Join  (cost=12.47..102.59 rows=53 width=576)
                 (actual time=0.430..2.067 rows=52 loops=1)
        Hash Cond: (c.doc_id = d.doc_id)
        ->  Seq Scan on chunks c  (cost=0.00..89.97 rows=53 width=64)
                                  (actual time=0.369..1.983 rows=52 loops=1)
              Filter: (word_count > 80)
              Rows Removed by Filter: 3546
        ->  Hash  (cost=11.10..11.10 rows=110 width=520)
                  (actual time=0.038..0.039 rows=38 loops=1)
              ->  Seq Scan on documents d  (cost=0.00..11.10 rows=110 width=520)
                                           (actual time=0.016..0.021 rows=38 loops=1)
Planning Time: 3.393 ms
Execution Time: 2.803 ms
```

Listing 3: Q1 EXPLAIN ANALYZE output — before indexing

Query 2 (Before):

```
Sort  (cost=24.96..25.05 rows=37 width=738)
      (actual time=0.207..0.210 rows=24 loops=1)
  Sort Key: d.year_published DESC
  ->  Hash Join  (cost=11.84..23.99 rows=37 width=738)
                 (actual time=0.072..0.186 rows=24 loops=1)
        Hash Cond: (co.country_id = d.country_id)
        ->  Seq Scan on countries co  (cost=0.00..11.30 rows=130 width=222)
```

```
                                    (actual time=0.014..0.017 rows=31 loops=1)
        ->  Hash  (cost=11.38..11.38 rows=37 width=524)
                  (actual time=0.036..0.037 rows=24 loops=1)
              ->  Seq Scan on documents d  (cost=0.00..11.38 rows=37 width=524)
                                           (actual time=0.015..0.024 rows=24 loops=1)
                    Filter: (year_published >= 2020)
                    Rows Removed by Filter: 14
Planning Time: 0.358 ms
Execution Time: 0.265 ms
```

Listing 4: Q2 EXPLAIN ANALYZE output — before indexing

6.4 Query Plans — After Indexes

Query 1 (After):

```
Sort  (cost=...) (actual time=... rows=52 loops=1)
  ->  Hash Join  (cost=6.55..54.70 rows=53 width=576)
                 (actual time=0.159..0.280 rows=52 loops=1)
        Hash Cond: (c.doc_id = d.doc_id)
        ->  Bitmap Heap Scan on chunks c  (cost=4.69..52.69 rows=53 width=64)
                                          (actual time=0.092..0.178 rows=52 loops=1)
              Recheck Cond: (word_count > 80)
              Heap Blocks: exact=23
              ->  Bitmap Index Scan on idx_chunks_word_count
                    (cost=0.00..4.68 rows=53 width=0)
                    (actual time=0.078..0.079 rows=52 loops=1)
                    Index Cond: (word_count > 80)
Planning Time: 4.437 ms
Execution Time: 0.443 ms
```

Listing 5: Q1 EXPLAIN ANALYZE output — after indexing

Query 2 (After):

```
Sort  (cost=13.80..13.83 rows=13 width=738)
      (actual time=0.101..0.104 rows=24 loops=1)
  ->  Hash Join  (cost=1.64..13.55 rows=13 width=738)
                 (actual time=0.069..0.084 rows=24 loops=1)
        ->  Seq Scan on documents d  (cost=0.00..1.48 rows=13 width=524)
                                     (actual time=0.012..0.018 rows=24 loops=1)
              Filter: (year_published >= 2020)
              Rows Removed by Filter: 14
Planning Time: 0.394 ms
Execution Time: 0.142 ms
```

Listing 6: Q2 EXPLAIN ANALYZE output — after indexing

7. Interpretation of Query Plans

Query 1 — `word_count` filter on `chunks`: Before indexing, PostgreSQL selected a sequential scan on `chunks`, reading all 3,598 rows and discarding 3,546 at the filter step — an I/O-intensive operation that constitutes the dominant cost at 2.803 ms. After creating `idx_chunks_word_count`, the planner adopted a two-phase Bitmap strategy. In the first phase (Bitmap Index Scan), it traversed the B-Tree index from the first entry satisfying `word_count > 80` and assembled a bitmap of 52 matching row pointers in 0.078–0.079

ms. In the second phase (Bitmap Heap Scan), it used the bitmap to fetch only 23 heap blocks (out of the full table), rechecked the predicate against those pages, and returned 52 rows. The total execution time fell to 0.443 ms, a **6.3× improvement**. The `Heap Blocks: exact=23` annotation confirms that no lossy (approximate) block-level reads were needed, indicating the working set fits within `work_mem`. This is the textbook outcome for a selective range predicate on a high-cardinality numeric column.

Query 2 — `year_published` filter on `documents`: The `documents` table has only 38 rows. For such a small table, PostgreSQL's cost model correctly determines that a single sequential page read is cheaper than the random I/O overhead of a B-Tree lookup, so the planner retained the sequential scan after indexing. However, the index caused a significant improvement in the planner's estimates: the estimated qualifying row count dropped from 37 to 13, and the total plan cost fell from 24.96 to 13.80 (a 45% reduction). This tighter estimate improved join ordering in the Hash Join with `countries`, resulting in a real execution time reduction from 0.265 ms to 0.142 ms (**1.9× improvement**). This result illustrates an important and often overlooked point: the presence of an index benefits the query planner's cost model — and therefore downstream plan choices such as join order and join method — even when the index itself is not used for the physical scan.

8. Stored Procedure: `log_query`

Purpose. The RAG pipeline must persist every user query, its generated answer, and the identity of the most relevant retrieved chunk for later audit, retrieval-quality analysis, and system debugging. The stored procedure `log_query` encapsulates this insertion as a single atomic database operation, preventing partial writes and ensuring that the timestamp reflects the exact moment of pipeline execution rather than application-layer clock drift.

Definition:

```
CREATE OR REPLACE PROCEDURE log_query(
    p_query_text   TEXT,
    p_answer_text  TEXT,
    p_top_chunk_id INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO queries (query_text, answer_text, top_chunk_id,
        created_at)
    VALUES (p_query_text, p_answer_text, p_top_chunk_id,
        CURRENT_TIMESTAMP);
END;
$$;
```

Listing 7: Stored procedure `log_query` definition

Parameters:

- `p_query_text` — the user's natural-language question as submitted to the pipeline.
- `p_answer_text` — the LLM-generated answer returned by the pipeline.
- `p_top_chunk_id` — the primary key of the highest-ranked retrieved chunk, linking the log entry to the specific evidence used.

How to run:

```
-- Insert a sample query log
CALL log_query(
    'What are Tanzania climate commitments?',
    'Tanzania committed to reduce emissions by 10-20% under its
       NDC.',
    4001
);

-- Verify the insertion
SELECT * FROM queries;
```

Listing 8: Calling and verifying `log_query`

Verification. The procedure was tested with `chunk_id = 4001` (a verified existing chunk in the live database). The subsequent `SELECT * FROM queries` confirmed a successful row insertion with the correct `query_text`, `answer_text`, and an auto-generated `created_at` timestamp of `2026-06-01 10:33:46`. The procedure uses `CURRENT_TIMESTAMP` rather than an application-supplied timestamp, ensuring that the audit log reflects server time and cannot be falsified by a misconfigured client clock.

9. Conclusions

Two targeted B-Tree indexes were added to the Climate Policy RAG database, addressing the highest-cost sequential scans identified in the Milestone 2 query workload. Query 1 achieved a confirmed 6.3× execution time improvement, with the query plan changing from a full sequential scan (3,598 rows read, 3,546 discarded) to a Bitmap Index Scan fetching only 23 heap blocks. Query 2 achieved a 1.9× improvement through a tighter planner cost model even without a scan strategy change, demonstrating that index benefits are not limited to cases where the index is directly used for the physical scan.

Together, these results validate two complementary aspects of B-Tree index utility in PostgreSQL: direct I/O reduction for selective range predicates on large tables, and cost-model improvement for small tables involved in multi-table joins. The stored procedure `log_query` provides a production-ready, atomic mechanism for persisting RAG pipeline query logs, supporting future retrieval-quality analytics and debugging workflows.

For the final submission, the next performance priority is migrating the `embedding TEXT`

column to a `pgvector VECTOR(384)` column and creating an HNSW or IVFFlat index to enable sub-linear approximate nearest-neighbour search — the most computationally expensive operation in the current pipeline.